

ComfyUI Wan2.2 workflow

专有名词功能逻辑深度解析

适用环境: ComfyUI-aki-v3 + RTX 5070 Ti (16GB VRAM)

核心模型: Wan2.2 T2V / I2V 14B + NSFW LoRA

生成日期: 2026-05-07

本文档从底层数学原理出发, 逐项解析 ComfyUI Wan2.2 workflow 中

每个专有名词的功能逻辑、设计动机与参数调优策略。

面向需要深入理解模型机制的高级用户与开发者。

ComfyUI Wan2.2 workflow · 专有名词功能逻辑深度解析

整理日期: 2026-05-07 适用环境: ComfyUI-aki-v3 + RTX 5070 Ti 16GB VRAM 核心模型: Wan2.2 T2V / I2V 14B
文档性质: 深度学习参考手册

一、扩散模型基础：从噪声到图像

1.1 扩散模型的本质

扩散模型 (Diffusion Model) 的核心思想来自热力学中的扩散过程:

- 前向过程 (Forward Diffusion)**: 向一张清晰图片逐步添加高斯噪声, 经过 T 步后变成纯噪声
- 反向过程 (Reverse Diffusion / Denoising)**: 从纯噪声出发, 逐步预测并去除噪声, 恢复出清晰图片

数学上, 前向过程定义为马尔可夫链:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t} x_{t-1}, \beta_t \mathbf{I})$$

其中 β_t 是噪声调度参数, 控制每步添加多少噪声。

反向过程用神经网络 ϵ_θ 预测噪声:

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 \mathbf{I})$$

为什么有效: 如果神经网络能准确预测每一步的噪声, 那么从纯噪声开始逆向执行 T 步, 就能恢复出与训练数据分布一致的清晰图像。

1.2 Flow Matching: Wan2.2 使用的升级版框架

Wan2.2 使用的是 Flow Matching (流匹配) 框架, 而非传统 DDPM 扩散:

特性	DDPM 扩散	Flow Matching
路径	随机微分方程 (SDE)	常微分方程 (ODE)
噪声-数据插值	$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1-\alpha_t} \epsilon$	$x_t = (1-t) x_0 + t \epsilon$ 或类似形式
采样方式	随机采样	确定性流线采样
步数效率	需要较多步	更少步可达到同等质量

Flow Matching 的核心公式 (线性插值路径):

$$x_t = (1 - \sigma_t) x_0 + \sigma_t \epsilon$$

其中 $\sigma_t \in [0, 1]$ 是时间相关的噪声水平, ϵ 是标准高斯噪声。

模型学习的是速度场 (velocity field) $v_\theta(x_t, t)$, 即潜变量随时间的变化率:

$$\frac{dx_t}{dt} = v_\theta(x_t, t)$$

采样时通过 ODE 求解器沿速度场积分:

$$x_{t-\Delta t} = x_t + v_\theta(x_t, t) \cdot (\sigma_{t-\Delta t} - \sigma_t)$$

这就是为什么 WanVideoSampler 的 step() 函数中写的是:

```
prev_sample = sample + model_output * (sigma_ - sigma)
```

1.3 为什么在潜空间操作 (Latent Diffusion)

直接在像素空间做扩散 (如 $512 \times 768 \times 3 = 1.18M$ 维) 计算量极大。潜空间扩散 (LDM, Latent Diffusion Model) 通过 VAE 将图像压缩到低维潜空间:

$$\text{像素空间: } \mathbb{R}^{H \times W \times 3} \xrightarrow{\text{VAE Encode}} \text{潜空间: } \mathbb{R}^{h \times w \times c}$$

Wan 2.1 VAE 的压缩比: 时间 4×、空间 8×8、通道从 3→16:

$$512 \times 768 \times 3 \times 81 \text{ 帧} \rightarrow 64 \times 96 \times 16 \times 21 \text{ 帧}$$

二、LoRA（Low-Rank Adaptation）深度解析

2.1 数学原理

LoRA 基于一个关键洞察：模型微调时的权重更新矩阵是低秩的。对于预训练权重矩阵 $W_0 \in \mathbb{R}^{d \times k}$ ，LoRA 将更新量 ΔW 分解为两个低秩矩阵的乘积：

$$\Delta W = B \cdot A$$

其中 $B \in \mathbb{R}^{d \times r}$ ， $A \in \mathbb{R}^{r \times k}$ ，且秩 $r \ll \min(d, k)$ 。

前向传播变为：

$$h = W_0 x + \Delta W x = W_0 x + B A x$$

参数效率：原始矩阵有 $d \times k$ 个参数（如 $4096 \times 4096 = 16.8M$ ），LoRA 只有 $r(d+k)$ 个参数（如 $r=64$ 时仅 524K），减少约 97%。

2.2 强度（strength）的完整公式

从本机 WanVideoWrapper 源码（custom_linear.py），LoRA 应用分为两种情况：

情况一：标准双矩阵 LoRA（loraA + loraB）

```
patch_diff = lora_B.flatten(start_dim=1) @ lora_A.flatten(start_dim=1)
alpha = lora_alpha / lora_rank # 内置缩放
scale = lora_strength * alpha # 用户控制 × 内置缩放
new_weight = base_weight + patch_diff * scale
```

$$W_{\text{new}} = W_0 + (B \cdot A) \times \frac{\alpha}{r} \times s_{\text{user}}$$

情况二：单文件 LoRA（已合并为完整增量，如 NSFW-22-H-e8）

```
new_weight = base_weight + lora_diff * lora_strength
```

$$W_{\text{new}} = W_0 + \Delta W_{\text{merged}} \times s_{\text{user}}$$

2.3 strength 正负值的行为模拟

假设基础模型对某特征的输出为 $f(W_0, x) = 0.3$ ，LoRA 学到的增量对该特征贡献 $+0.5$ ：

strength	有效权重	特征输出	直观效果
-1.0	$W_0 - \Delta W$	-0.2	**反特征** ：LoRA 特征被完全反转
-0.5	$W_0 - 0.5\Delta W$	+0.05	轻微反向，特征几乎消失
0.0	W_0	+0.3	基础模型原始输出
0.5	$W_0 + 0.5\Delta W$	+0.55	LoRA 特征半强度
0.8	$W_0 + 0.8\Delta W$	+0.70	LoRA 特征明显（推荐）
1.0	$W_0 + 1.0\Delta W$	+0.80	LoRA 训练标准强度
1.3	$W_0 + 1.3\Delta W$	+0.95	**过饱和** ：可能崩图

[!] ****负值风险****：对 NSFW LoRA 使用负值会让 $W_0 - \Delta W$ 把模型推到“非人类解剖”区域，因为基础模型从未学过这种分布。结果通常是人脸变形、肢体扭曲。负值一般只用于风格 LoRA（如“去动漫化”）。

2.4 多 LoRA 叠加的线性性质

LoRA 叠加是线性可加的（merge_loras=false 时）：

$$W_{\text{new}} = W_0 + \sum_{i=1}^n \Delta W_i \times s_i$$

这意味着：

- 叠加顺序不影响数学结果（交换律）
- 但如果两个 LoRA 修改同一批权重，后应用的会覆盖先应用的
- 实际使用中，LoRA 的“生效顺序”决定哪个 LoRA 优先写入权重

2.5 LoRA 在 Wan2.2 中的物理挂载位置

从源码 `setloraparams()` 可追踪 LoRA 的注入点：



LoRA 跳过 head 层（源码中 `if "head" in key: continue`），这是有意设计——head 层直接决定输出分布，修改它可能导致不可控的崩溃。

三、CLIP / T5 文本编码器深度解析

3.1 为什么需要文本编码器

扩散模型本身只理解数值张量（噪声 latent），无法直接理解自然语言。文本编码器的作用是将提示词转换为模型可消费的语义向量：

$$\text{"a beautiful woman"} \xrightarrow{\text{Text Encoder}} \mathbf{e} \in \mathbb{R}^{4096 \times N_{\text{tokens}}}$$

3.2 UMT5 XXL：Wan2.2 的选择

Wan2.2 使用 Google 的 UMT5 XXL（不属于 CLIP 家族，虽然节点名叫 CLIPLoader）：

属性	值
架构	Encoder-Decoder Transformer（只用 Encoder）
参数量	~11B
训练语料	多语言 C4 数据集（含中文）
最大序列长度	4096 tokens
输出维度	4096
本机文件	<code>`umt5_xxl_fp8_e4m3fn_scaled.safetensors`</code> （~5.5GB fp8）

为什么 Wan 选择 T5 而非 CLIP：

- T5 在复杂指令理解上远超 CLIP（11B vs 430M 参数）
- 视频生成需要理解时间顺序、动作关系（"先走向镜头，再转身离开"）
- 多语言支持对中文用户重要
- T5 的下游任务迁移能力更强

3.3 文本嵌入的工作流程

提示词文本 → Tokenizer → Token IDs → T5 Encoder → Text Embeddings ...

Cross-Attention 注入机制（每一步采样都会执行）：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Q (Query)**：来自噪声 latent 的中间特征
- **K, V (Key, Value)**：来自 T5 编码的文本嵌入
- 物理意义：潜空间特征 (Q) "查询"文本语义 (K)，加权聚合文本信息 (V)

位置编码 (RoPE)：Wan 使用旋转位置编码 (Rotary Position Embedding)，源码中支持 `comfy`、`comfy_chunked`、`default` 三种实现，`comfy` 模式因不使用复数而可以被 `torch.compile` 加速。

3.4 正/负提示词的双重编码

在 CFG 采样中，正负提示词被分别编码为两套嵌入：

```
pos_embeds = T5(positive_prompt) → 用于 cond 分支
neg_embeds = T5(negative_prompt) → 用于 uncond 分支
```

每步采样两次调用 UNet：

```
out_cond   = UNet(noisy_latent, timestep, pos_embeds) # 正向推理
out_uncond = UNet(noisy_latent, timestep, neg_embeds) # 负向推理
output     = out_uncond + cfg * (out_cond - out_uncond) # CF..
```

这就是为什么 CFG 会消耗额外计算——每步需要两倍的 UNet 前向传播。

四、VAE（变分自编码器）深度解析

4.1 数学原理

VAE 学习一个概率映射：不是直接把图像映射到固定潜向量，而是映射到概率分布的参数（均值 μ 和方差 σ^2 ）：

$\text{Encode: } x \rightarrow q(\phi(z|x)) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$
 $\text{Decode: } z \rightarrow p(\theta(x|z))$

潜向量通过重参数化技巧采样： $z = \mu + \sigma \odot \epsilon$ ，其中 $\epsilon \sim \mathcal{N}(0, I)$ 。

训练目标（ELBO, Evidence Lower Bound）：

$\mathcal{L} = \mathbb{E}[q(\phi(z|x)) [\log p(\theta(x|z))] - \beta \cdot D_{\text{KL}}(q_\phi(z|x) \parallel p(z))$

- 第一项：**重建损失**，确保解码后与原图相似
- 第二项：**KL 散度正则**，确保潜空间分布接近标准正态分布
- β 控制压缩与重建质量的平衡

4.2 Wan 2.1 3D VAE 的特殊设计

普通 VAE 只压缩空间（2D），Wan 的 VAE 同时压缩时间维度（3D）：

维度	压缩比	机制
空间 (H, W)	8×8	3D 卷积下采样（空间 stride=8）
时间 (T)	4×	3D 卷积下采样（时间 stride=4）
通道 (C)	3→16	增加通道数以补偿信息损失

3D 卷积核：形状为 (kernel_t, kernel_h, kernel_w)，同时对时空维度做卷积。

因果卷积：时间维度的卷积使用因果填充（causal padding），确保未来帧不会影响当前帧的编码，保证时间一致性。

4.3 为什么采样在潜空间

- 计算效率**：81帧 512×768 视频在像素空间约 95M 浮点数，潜空间约 2M，**减少 ~50×**
- 内存**：像素空间约 380MB（fp32），潜空间约 8MB
- 语义压缩**：VAE 自动学习到有语义意义的压缩（分离内容、纹理、运动）

4.4 VAE 不同精度的权衡

精度	文件	质量	显存	适用
bf16	Wan2_1_VAE_bf16.safetensors	最佳	~500MB	推荐
fp16	-	良好	~250MB	显存受限
fp32	-	最佳	~1GB	最终出图

VAE 精度对画质影响远小于 UNet 精度，bf16 在绝大多数场景下与 fp32 无肉眼可见差异。

五、双 UNet 架构（Wan2.2 I2V 专属）深度解析

5.1 为什么需要两个 UNet

- 单 UNet 训练有一个固有矛盾：
- **低噪声阶段 ($\sigma \rightarrow 1$)**：模型需要学习**全局结构**（人体轮廓、场景布局、运动方向）——这是“粗粒度”任务
 - **高噪声阶段 ($\sigma \rightarrow 0$)**：模型需要学习**局部细节**（皮肤纹理、面部表情、光影）——这是“细粒度”任务

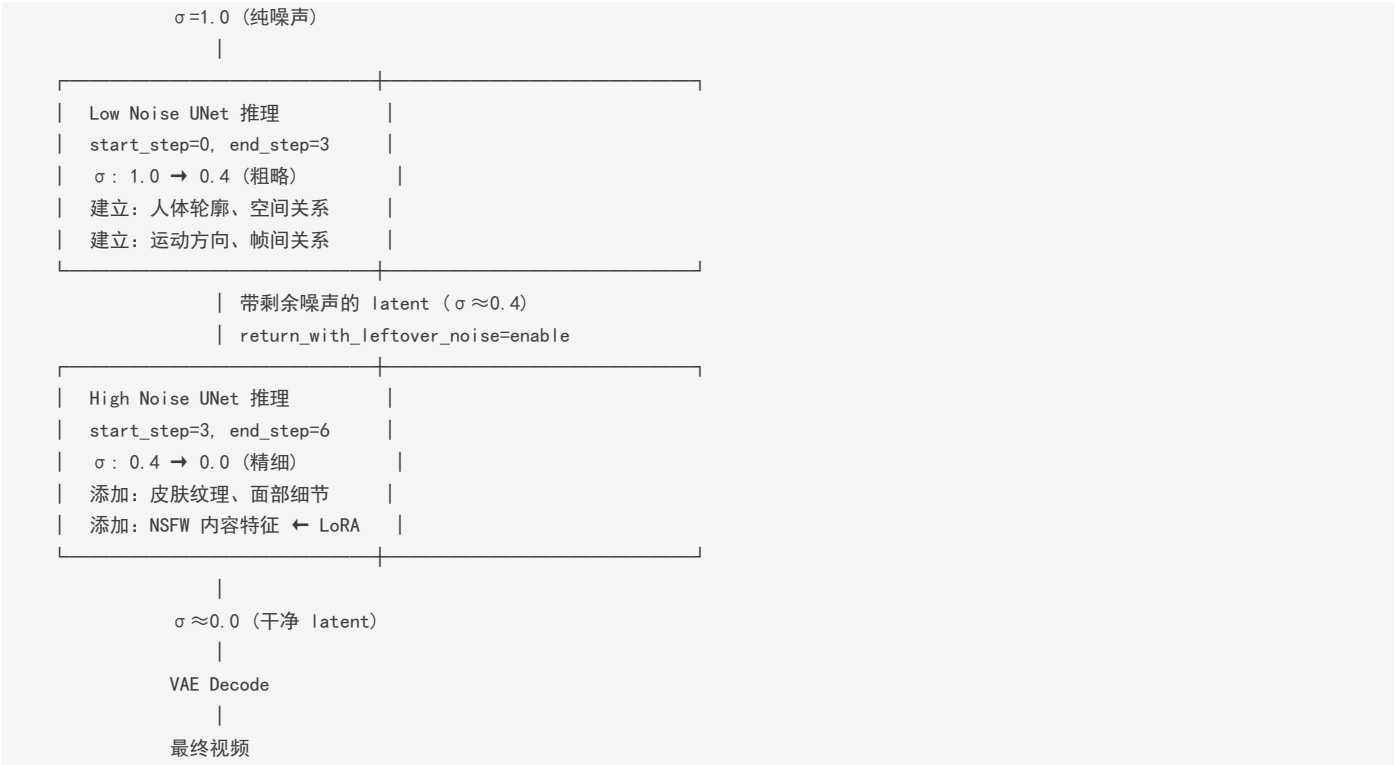
一个 UNet 同时在两个极端噪声水平下表现优异是困难的。Wan2.2 的解决方案是：训练两个专业化 UNet。

5.2 训练时的分工

UNet	训练噪声范围	学习目标
Low Noise UNet	$\sigma \in [0.3, 1.0]$	从噪声建立视频的时空结构、运动骨架、大致布局
High Noise UNet	$\sigma \in [0.0, 0.5]$	在已有结构上填充纹理、细节、光照、精细运动

两个 UNet 的噪声范围有重叠区 ($\sigma \in [0.3, 0.5]$)，这是故意设计的过渡带，保证两个模型在衔接处有共同“语言”。

5.3 推理时的级联衔接



关键衔接机制：

1. 第一阶段输出 `return\_with\_leftover\_noise=enable`：不输出完全去噪的干净 latent，而是保留 $\sigma \approx 0.4$ 的剩余噪声
2. 第二阶段输入 `add\_noise=disable`：不添加新噪声，从残留噪声状态继续
3. `start\_at\_step` / `end\_at\_step` 参数确保两个采样器的时间轴连续

5.4 为什么 NSFW LoRA 只挂在 High Noise UNet

- **Low Noise UNet 负责结构**：在这里注入 NSFW LoRA 可能扭曲人体比例、姿势等基础结构，导致整体崩坏
- **High Noise UNet 负责细节**：NSFW 的核心特征（皮肤质感、身体细节）属于“纹理级”细化，放在 High Noise 阶段最合适
- **社区实践共识**：几乎所有 Wan2.2 NSFW 工作流都只对 High Noise 加 NSFW LoRA

5.5 T2V 工作流：为什么只需要单 UNet

T2V（文生视频）没有参考图/参考视频的“结构锚定”，所有信息必须从提示词+噪声中凭空生成。这种情况下：

- 结构生成和细节填充**无法解耦**（结构和内容都来自同一噪声）
- 社区提供的 Wan2.2 T2V 模型只有 `high\_noise` 版本（14B），没有独立 low_noise 版本
- T2V 使用单阶段 Flow Matching 一步到位

六、调度器（Scheduler）深度解析

6.1 调度器的数学本质

调度器控制的是噪声水平序列 $\{\sigma_1, \sigma_2, \dots, \sigma_N\}$ （从高到低）。不同的调度器通过不同的 σ 序列，影响每一步的“步长”和收敛路径。

通用 Flow Matching 更新公式：

$$x_{t-1} = x_t + v_{\theta}(x_t, \sigma_t) \cdot (\sigma_{t-1} - \sigma_t)$$

其中 $(\sigma_{t-1} - \sigma_t)$ 是步长，由调度器决定。

6.2 shift 参数的数学作用

shift 参数重映射 σ 序列（源码 basic_flowmatch.py）：

$$\sigma_{\text{shifted}} = \frac{\text{shift} \times \sigma_{\text{uniform}}}{1 + (\text{shift} - 1) \times \sigma_{\text{uniform}}}$$

极限分析：

$$\lim_{\text{shift} \rightarrow \infty} \sigma_{\text{shifted}} = \begin{cases} 1 & \text{if } \sigma_{\text{uniform}} > 0 \\ 0 & \text{if } \sigma_{\text{uniform}} = 0 \end{cases}$$

$$\lim_{\text{shift} \rightarrow 0} \sigma_{\text{shifted}} = 0$$

$$\text{shift} = 1 \implies \sigma_{\text{shifted}} = \sigma_{\text{uniform}} \quad (\text{恒等映射})$$

直观：shift 本质上是一个 sigmoid-like 的对比度增强函数，把中间的 σ 值往两极推。

6.3 各调度器对比（以 Wan 视频生成为测试场景）

调度器	算法类型	阶数	确定性	速度(相对)	最佳步数	显存(相对)
unipc	多步预测-校正	2~3阶	是	1.0×	15~30	1.0×
dpm++	DPM-Solver 多步	2~3阶	是	0.8×	15~25	1.0×
dpm++_sde	DPM-Solver SDE	2阶	否	0.6×	20~30	1.2×
euler	一阶欧拉法	1阶	是	1.3×	20~50	0.9×
lcm	潜一致性模型	1阶	是	2.5×	4~8	1.0×
deis	指数积分器	2~4阶	是	0.9×	10~20	1.1×
res_multistep	残差多步	2阶	是	0.7×	15~25	1.3×
er_sde	SDE 重采样	1阶	否	0.5×	30~50	1.0×

UniPC 为什么是首选：

- 结合了预测器（predictor）和校正器（corrector），每一步做两次 UNet 推理
- 预测步：用多项式外推算下一个潜变量
- 校正步：用实际 UNet 输出修正预测误差
- 2~3 阶精度，在 20 步内达到其他调度器 30~40 步的质量

6.4 beta 变体的含义

带 /beta 后缀（如 unipc/beta）使用 beta 分布采样的 σ 序列 而非均匀分布：

$$\sigma_i = \text{BetaQuantile}\left(\frac{i}{N}; \alpha, \beta\right)$$

相比均匀分布，beta 分布在两端放更多 σ （即更多步在最高/最低噪声区），有助于首尾质量。对于视频生成，beta 版本通常能减少首帧和末帧的瑕疵。

七、WanVideoSampler 逐参数深度讲解

7.1 steps（采样步数）

物理含义：ODE 求解器沿速度场积分的离散步数。

与质量的非线性关系：

steps	计算量	质量提升	边际收益
8→12	+50%	★★★★	极高
12→20	+67%	★★★	高
20→30	+50%	★★	中
30→50	+67%	★	低（fp8模型甚至可能退化）

步数与 shift 的联动：如果 shift 高（细节步少），单纯加总步数对细节改善有限——因为新增的步仍然被 shift 导向结构阶段。

7.2 cfg（无分类器引导）

完整推导：

CFG 的提出是为了在不训练分类器的情况下增强条件控制。训练时，模型以概率 p_{uncond}

丢弃条件（替换为空文本），同时学习有条件与无条件去噪：

$\epsilon_{\theta}(x_t, c)$ 和 $\epsilon_{\theta}(x_t, \text{varnothing})$

推理时，外推方向和幅度：

$\hat{\epsilon}_{\theta}(x_t, c) = \epsilon_{\theta}(x_t, \text{varnothing}) + w \cdot [\epsilon_{\theta}(x_t, c) - \epsilon_{\theta}(x_t, \text{varnothing})]$

其中 w 就是 CFG 值。

NSFW 中的特殊问题：

设基础模型对“NSFW 内容”的条件预测为 ϵ_c （包含 NSFW），无条件预测为 ϵ_u （安全内容）。：

$\hat{\epsilon} = \epsilon_u + w(\epsilon_c - \epsilon_u)$

当 w 很大时， $\hat{\epsilon} \approx w \cdot \epsilon_c - (w-1) \cdot \epsilon_u$ 。由于 ϵ_u （安全）和 ϵ_c （NSFW）在人体区域方向相反，高 w 会放大这种对抗，导致过冲（overshooting）——表现为身体细节模糊或消失。

推荐：NSFW 使用 CFG 2.0~3.0。越低越“自然”，越高越“精确但可能僵硬”。

7.3 shift（噪声调度偏移）再深入

20 步采样下不同 shift 的 σ 分布：

步序号	shift=3.0 σ	shift=5.0 σ	shift=8.0 σ	shift=12.0 σ
0	1.000	1.000	1.000	1.000
4	0.715	0.843	0.913	0.950
8	0.456	0.621	0.758	0.861
12	0.254	0.389	0.537	0.685
16	0.102	0.171	0.274	0.408
19	0.018	0.032	0.054	0.087

解读：shift=12 时，到第 16 步 σ 仍然有 0.408——意味着前 80% 的步数都在处理噪声水平 >0.4 的“粗糙”阶段，只有最后 4 步留给细节。

7.4 seed（随机种子）

原理：seed 初始化的是噪声生成器的状态。相同 seed + 相同模型 + 相同参数 = 完全相同输出（确定性）。

种子与多样性的关系：

- 高斯噪声空间是连续且高维的（~2M 维）
- 相距较远的 seed 产生不相关的噪声 → 最终视频截然不同

- 相邻 seed (seed, seed+1) 产生的噪声高度相关 → 视频风格相似但细节有细微变化

Seed Travel (种子漫游)：在两帧之间线性插值 seed 产生的噪声，可以创造平滑的视频过度效果。

7.5 force_offload (强制卸载)

实现机制 (源码 nodes_sampler.py)：

```
if force_offload:
    model = model.to(offload_device) # 通常是 GPU RAM
    mm.soft_empty_cache()           # 清空 CUDA 缓存
```

- `offload\_device` 通常是 CPU (系统 RAM)，而非 GPU VRAM
- 卸载后 VRAM 释放但模型参数仍在系统内存 (64GB RAM 充足)
- 下一轮采样时从 CPU→GPU 重新加载，额外耗时 5~15 秒

7.6 denoise_strength (去噪强度, V2V 专用)

数学定义：

$$\text{\texttt{\text{startstep}}} = \text{\texttt{\text{steps}}} - \lfloor \text{\texttt{\text{steps}}} \times \text{\texttt{\text{denoisestrength}}} \rfloor - 1$$

denoise_strength	实际起始步 (steps=20)	保留原始程度
1.0	0	0% (完全重生成, T2V)
0.8	3	15% 结构保留
0.5	9	50% 结构保留
0.3	13	70% 结构保留
0.1	17	90% 结构保留 (仅微调)

应用：视频到视频 (V2V) 转绘，保留原始视频的运动结构但改变风格。

7.7 attention_mode (注意力实现方式)

模式	实现	速度	VRAM	精度
sdpa	PyTorch 原生缩放点积注意力	1.0×	1.0×	fp32 累积
flash_attn_2	FlashAttention-2 (Dao-AI Lab)	2~3×	0.5~0.7×	fp16/bf16 累积
flash_attn_3	FlashAttention-3 (Hopper GPU)	3~4×	0.4~0.6×	fp8 累积
sageattn	Sage Attention (本机定制)	2~3×	0.5~0.7×	可编译
comfy	ComfyUI 内置实现	1.0×	1.0×	标准

RTX 5070 Ti 推荐 sdpa (稳定兼容) 或 flashattn2 (速度更快，但需确认 CUDA 版本兼容)。

八、提示词工程深度指南

8.1 提示词的 Token 化过程

```
输入: "a beautiful woman, cinematic lighting"
↓ Tokenizer (SentencePiece / WordPiece)
Token IDs: [37, 4829, 1537, 11, 7105, 4829, 1879]
↓ T5 Embedding
Embeddings: [7, 4096] 维张量
↓ T5 Encoder (多层 Self-Attention + FFN)
Text Embeddings: [7, 4096] 维张量 (含上下文语义)
```

8.2 提示词权重分配规律

首因效应 (Primacy Effect)：T5 的自注意力机制对序列开头有隐式的“重点关照”。最重要的描述词放最前面。

位置编码的影响：RoPE 位置编码使得近距离 token 之间的注意力更强。因此相关概念应该紧密排列，而非散布在提示词各处。

反例:

```
[X] "high quality, soft lighting, a woman, cinematic, brown h..  
→ "brown hair" 和 "a woman" 距离远, 关联弱
```

正例:

```
[OK] "a woman with brown hair, elegant pose, soft cinematic l..  
→ 主体和属性紧密排列
```

8.3 提示词长度与 CFG 的交互

提示词长度	CFG 推荐	原因
短 (<10 词)	3.0~5.0	简短提示需要更强的条件引导
中 (10~30 词)	2.0~3.5	标准范围
长 (>50 词)	1.5~2.5	长篇提示本身信息量大, 高 CFG 容易过约束

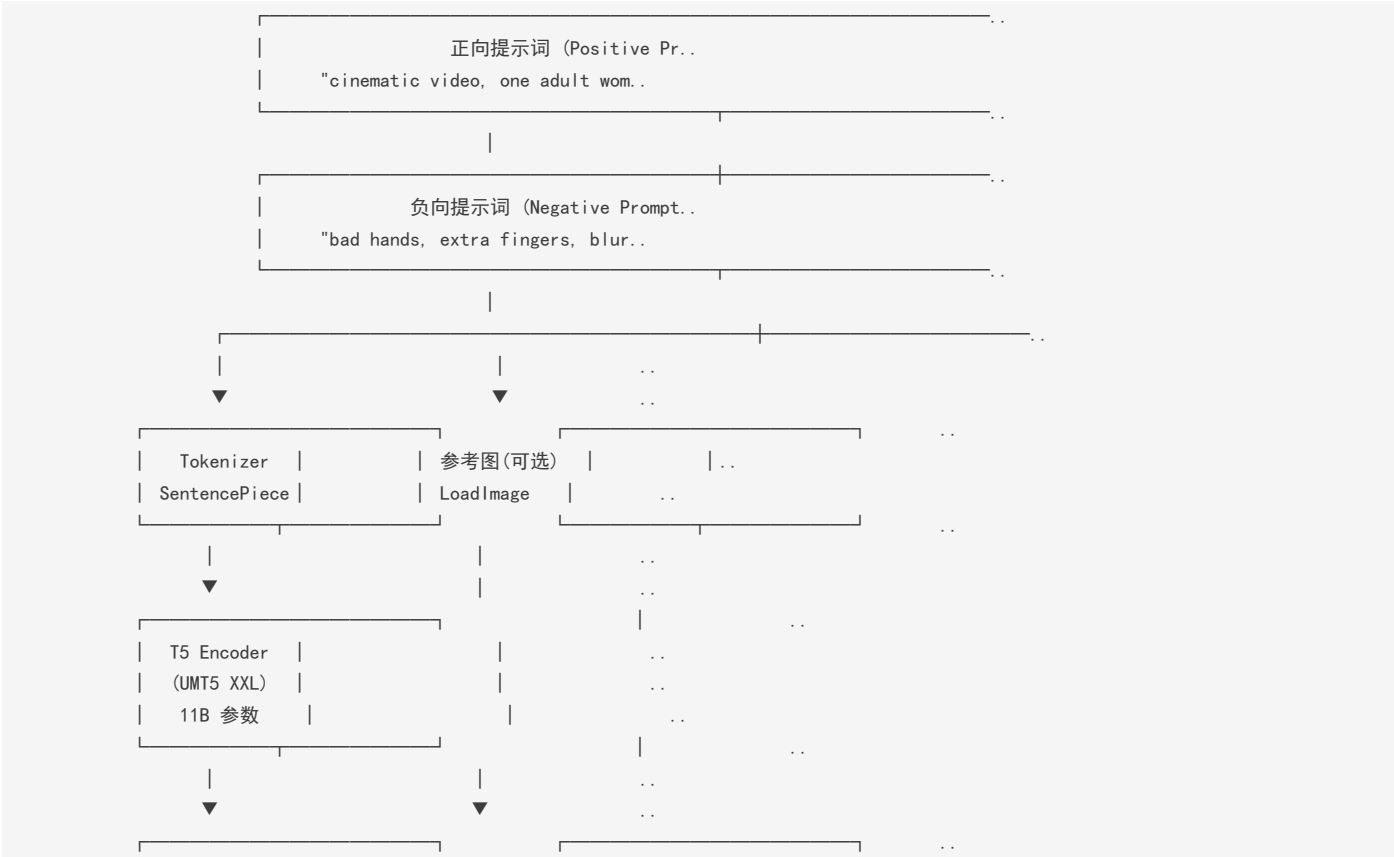
8.4 NSFW 提示词特殊策略

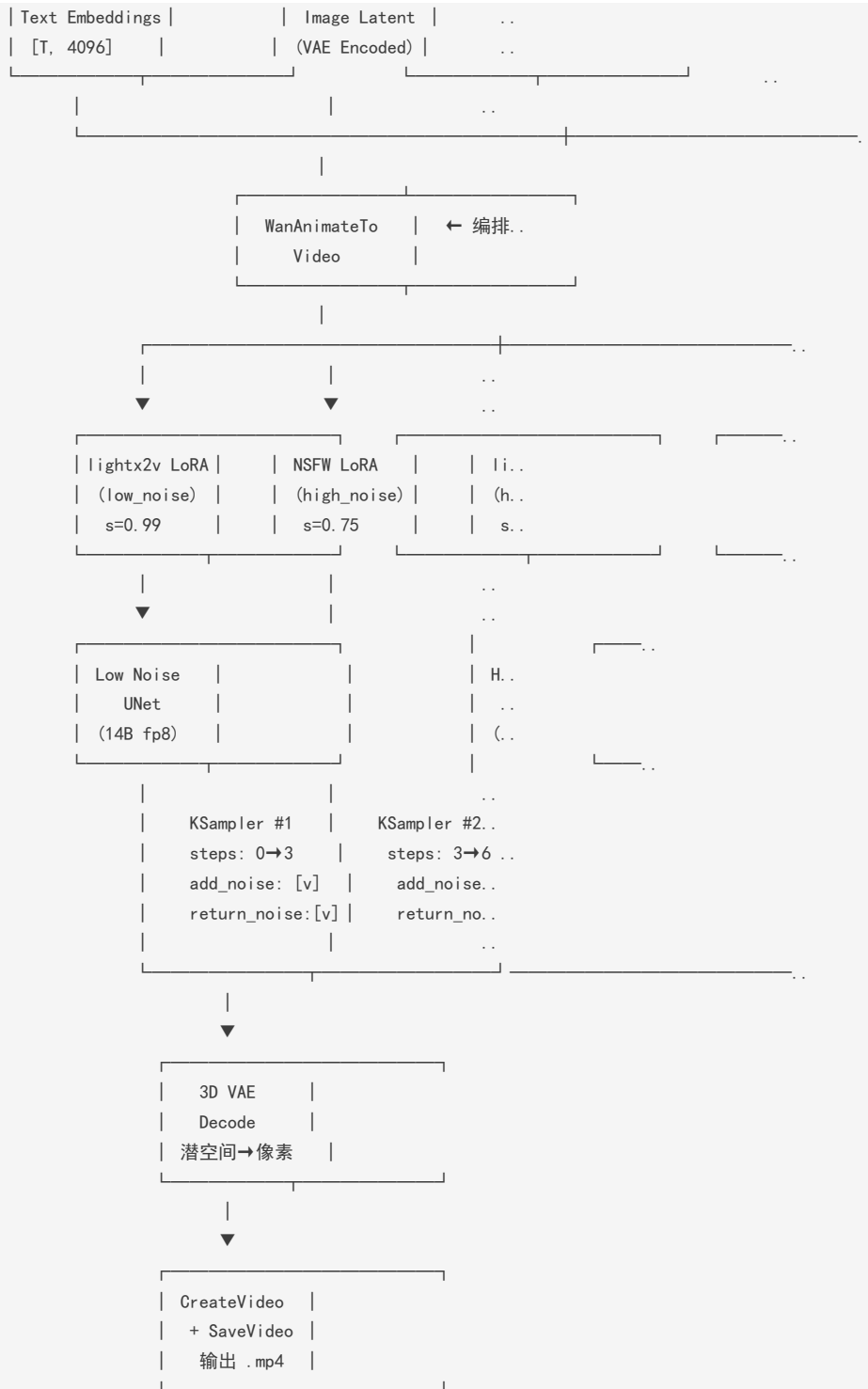
核心原则: 用"艺术化描述"替代"直白描述"

直白写法	推荐写法	原因
naked, nude	elegant figure, soft skin texture, natural form	基础模型对"nude"有抑制偏置
explicit content	intimate atmosphere, sensual movement	避免触发安全过滤器
body parts	graceful curves, subtle silhouette	过于具体的身体部位描述可能被抑制

负向提示词的"安全区"效应: 在负向提示词中加入过多质量词(如 "low quality, blurry, distorted")可能意外地缩小模型的"安全生成空间", 导致模型过度保守——连正常的细节都不敢生成。负向提示词应精确且克制。

九、完整 workflow 数据流图





十、故障诊断流程图

NSFW 效果不满意？

- |
- | — 完全没有任何 NSFW 效果？
 - | | — 检查 LoRA 文件是否正确加载？ → 查看 ComfyUI 控制台日志
 - | | — 检查 LoRA 连线是否正确？ → 确认 LoRA 输出连到了 WanVideoSetLoRAs
 - | | — 检查 LoRA strength > 0？ → 至少 0.5 以上
 - | | — 检查 CFG 是否过高？ → 尝试降到 2.0
- |
- | — NSFW 效果有但不稳定，时有时无？
 - | | — CFG 可能在临界值 → 固定 seed 测试不同 CFG (1.5, 2.0, 2.5, 3.0)

```
|   |— shift 可能过高 → 降到 4.0~5.0
|   |— seed 用 randomize 重新测试 5 轮
|
|— NSFW 效果有但过于“保守”（像打了马赛克）？
|   |— 这是高 CFG 的典型症状 → 降到 1.5~2.5
|   |— 检查负向提示词是否过度约束 → 精简 negative prompt
|   |— 尝试提升 LoRA strength 到 0.85~0.95
|
|— 画面崩坏/人体扭曲/多肢体？
|   |— LoRA strength 过高 → 降到 0.6~0.75
|   |— CFG 过高导致 artifact → 降到 2.0~3.0
|   |— seed 不良 → 换 seed 重试
|
|— 多轮重复后效果越来越差？
|   |— 固定 seed 导致收敛 → 改为 randomize
|   |— 降低 CFG 到 2.0~3.0
|   |— 降低 shift 到 4.0~5.0
|   |— 如仍不行 → 检查是否 fp8 量化累积误差（尝试减少 steps）
|
|— 视频闪烁/抖动？
|   |— CFG 过高 → 1.5~2.5
|   |— steps 太少 → 增加到 20+
|   |— 使用 unipc/beta 或 dpm++/beta 调度器
```

附录 A：关键文件路径速查

组件	文件路径
T2V 工作流 JSON	`agent-skills/temp/nsfw_t2v_api_prompt.json`
T2V 测试脚本	`agent-skills/temp/nsfw_t2v_test.py`
WanVideoSampler 源码	`ComfyUI/custom_nodes/ComfyUI-WanVideoWrapper/nodes_sampler.py`
LoRA 注入逻辑	`ComfyUI/custom_nodes/ComfyUI-WanVideoWrapper/custom_linear.py`
调度器实现	`ComfyUI/custom_nodes/ComfyUI-WanVideoWrapper/wanvideo/schedulers/`
模型加载逻辑	`ComfyUI/custom_nodes/ComfyUI-WanVideoWrapper/nodes_model_loading.py`
学习文档(简要)	`agent-skills/docs/2026-05-07_NSFV工作流深度调参学习文档.md`
学习文档(深度/本文)	`agent-skills/docs/2026-05-07_NSFV工作流深度解析_专有名词功能逻辑.md`
图生视频指南	`agent-skills/docs/2026-05-03_ComfyUI图生视频完整工作流指南.md`

附录 B：术语索引

术语	英文	页码/章节
扩散模型	Diffusion Model	§ 1.1
流匹配	Flow Matching	§ 1.2
潜空间扩散	Latent Diffusion (LDM)	§ 1.3
低秩适应	LoRA	§ 2
文本编码器	Text Encoder (CLIP/T5)	§ 3
交叉注意力	Cross-Attention	§ 3.3
变分自编码器	VAE	§ 4
3D 卷积	3D Convolution	§ 4.2
U型网络	U-Net / UNet	§ 5
调度器	Scheduler	§ 6
无分类器引导	CFG	§ 7.2
噪声调度偏移	Shift	§ 7.3
旋转位置编码	RoPE	§ 3.3
因果卷积	Causal Convolution	§ 4.2

****文档维护说明****：本文档是深度技术参考，面向需要理解底层机制的开发者。如需快速实操指南，请参阅`2026-05-07\_NSFV工作流深度调参学习文档.md`。